# Block Volume Calculation

**Block Volume Calculation**

1. Course Content
2. Preparation
3. Case Demonstration
    Color Calibration
4. Source Code Analysis

## 1. Course Content

- After that, when you select a color block, the program will recognize the shape of the selected color block, obtain the relevant data of the color block, and calculate its volume.

> **Important**
>
> Only depth camera users should refer to this section

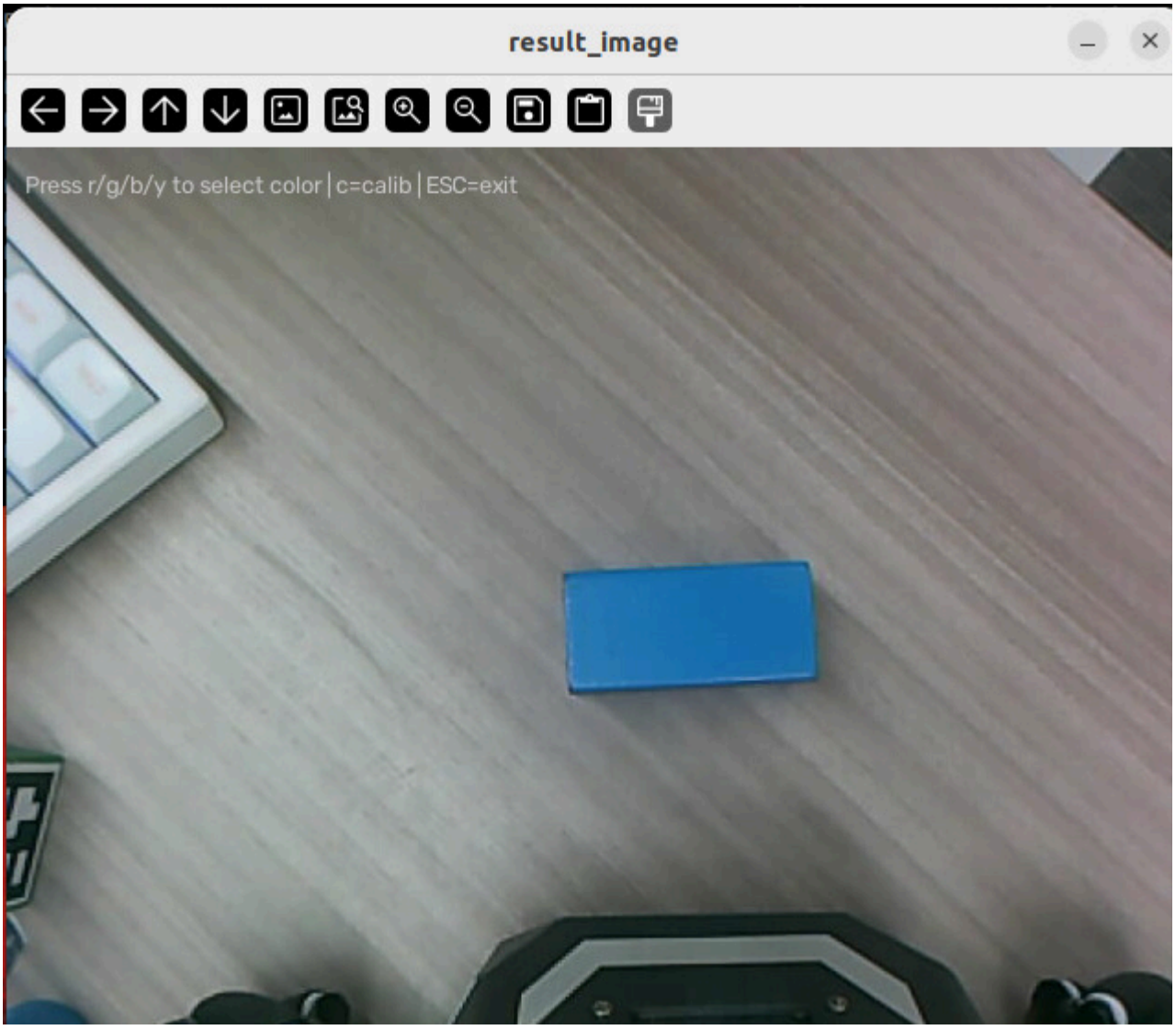
## 2. Preparation

- Demonstration prop: 30 * 30 * 60mm colored wooden block



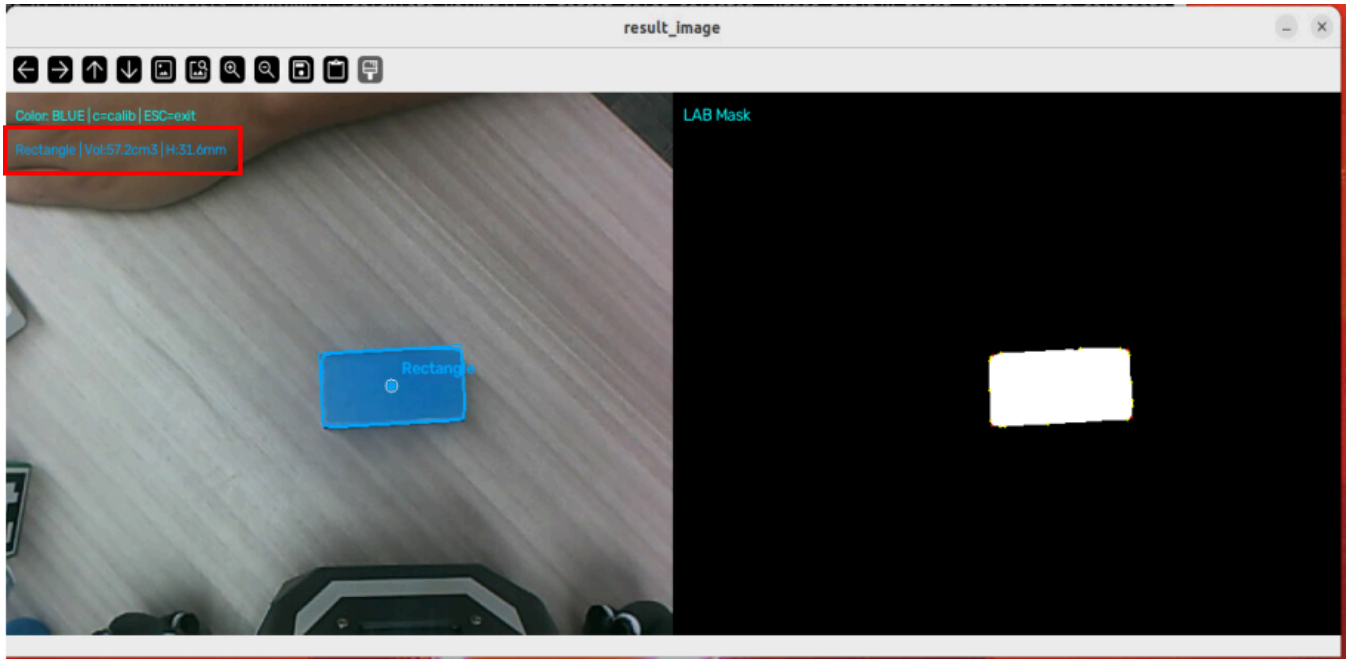
- Start the program

```
ros2 launch yahboomcar_bringup camera_demo.launch.py demo:=calculate_volume
```




## 3. Case Demonstration

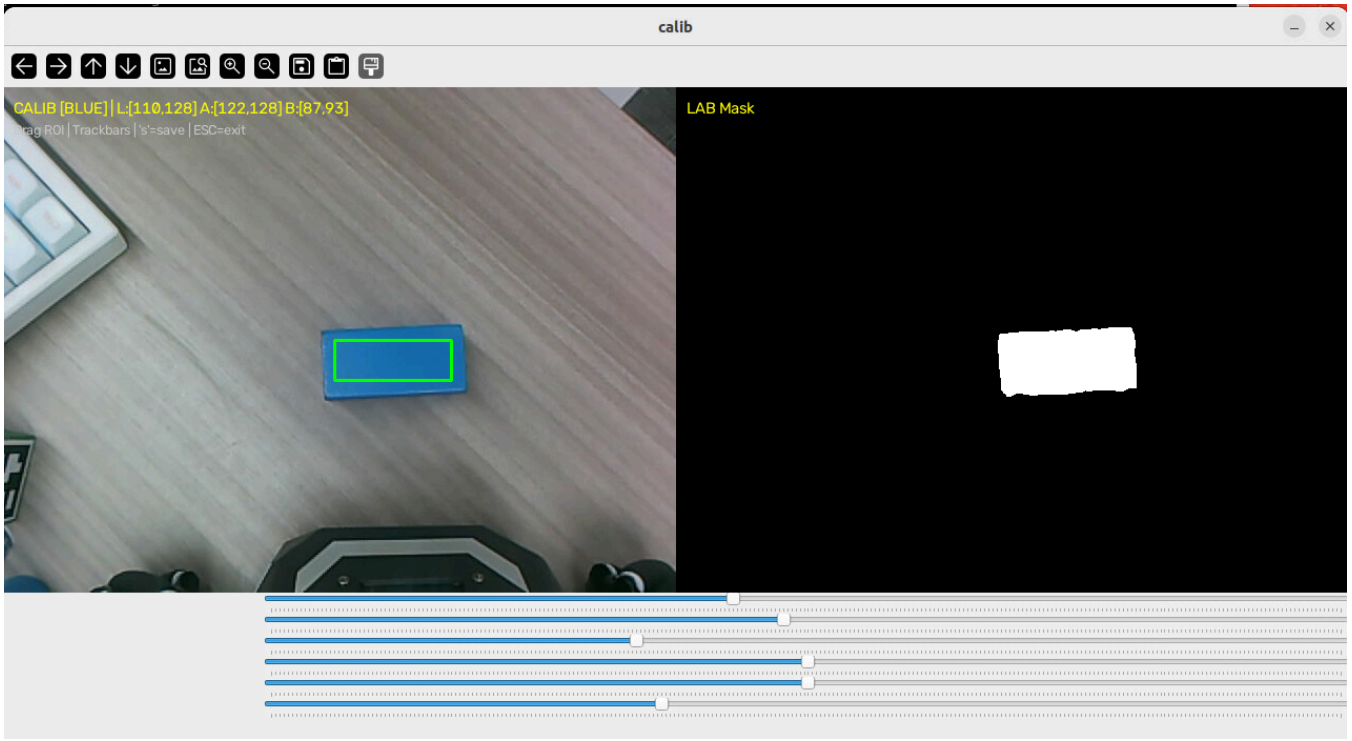
- After the program runs, press r (red), g (green), b (blue), or y (yellow) on the window to select the color to detect



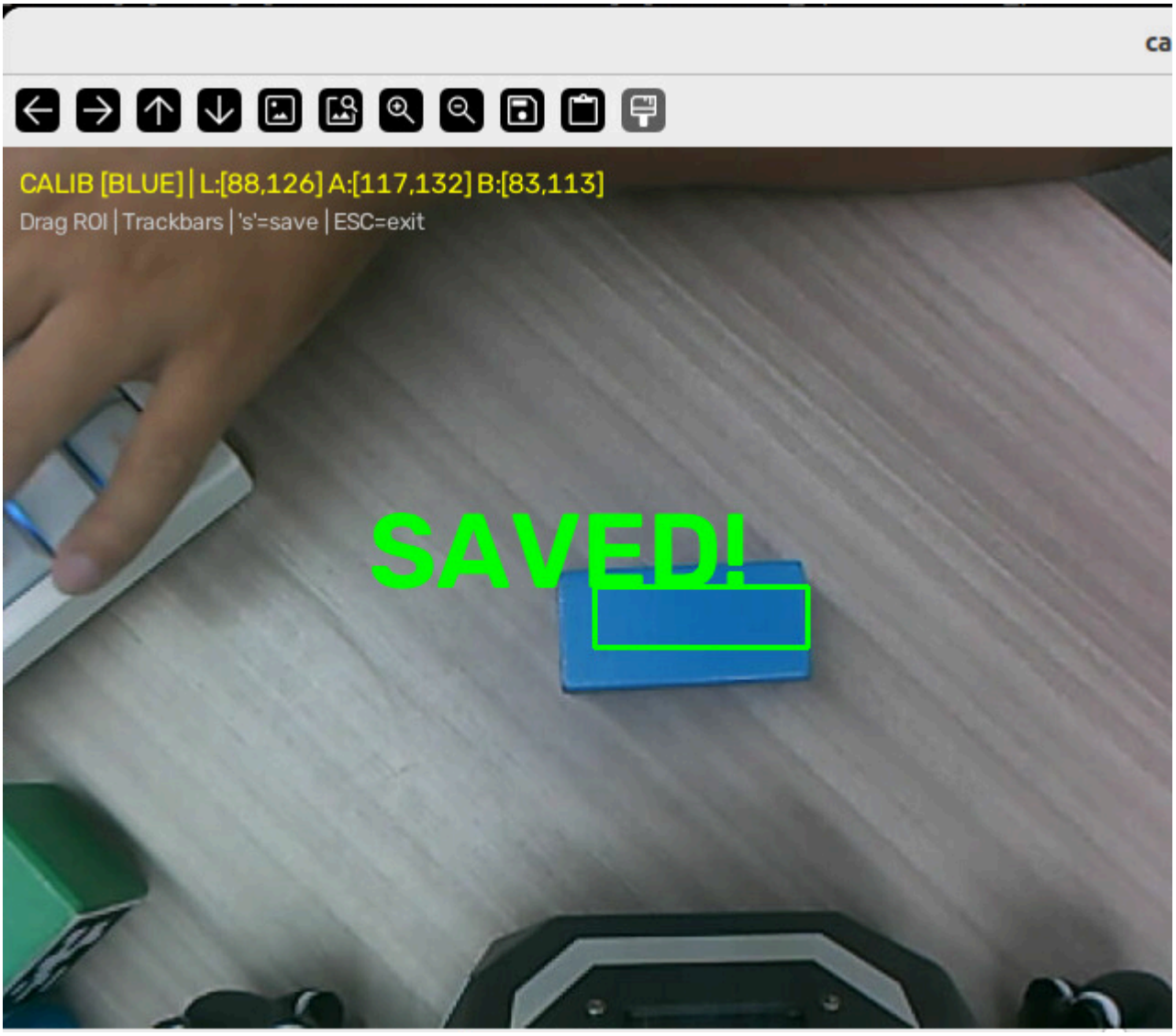
- The program can detect three shapes of wooden blocks: square, circle, and rectangle
- The program uses LAB values to filter out other colors and keeps only the colors within the HSV value range. The program recognizes the shape of the color block and calculates the color block's data based on its center point, and then calculates the color block's volume based on the recognized shape. The estimated volume and the height of the wooden block's center point are displayed in the upper-left of the window, as shown above.

## Color Calibration

- If the color block is not recognized well, press `c` on the keyboard in the window to enter color calibration. Press and hold the left mouse button and drag to select the target region's color; it is fine as long as the LAB Mask shape on the right is relatively complete. You can fine-tune the LAB detection values by sliding the slider below to the left or right




- Press the `s` key to save the calibration result, then press ESC to exit calibration




## 4. Source Code Analysis

- Path of the source code `calculate_volume.py`:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/camera/calculate_volume.py
```

- `_rgb_callback`: RGB image callback, the core processing flow
  - Get the synchronized depth image (thread-safe) and convert it to OpenCV format
  - Calibration mode: press `c` to enter the LAB calibration UI, press ESC to exit calibration
  - Color selection: `r/g/b/y` keys switch the target color and clear the tracked color blocks
  - Call `_detect_shapes_by_color` to detect shapes through the color mask
  - Call `_stabilize_blocks` for temporal stability processing to eliminate flicker between frames
  - For each detected color block, call `_calculate_volume` to calculate the volume and height
  - Call `_draw_results` to draw the detection results

- `_draw_results`: drawing function
  - Shape and display color mapping: Square=blue, Rectangle=orange, Circle=green
  - Draw the contour, the minimum-area rectangle box, the center point, and the shape label
  - Display the volume and height information of each color block in the upper-left
  - Display the LAB mask image side by side with the result image

```
    def _rgb_callback(self, color_frame):
        if self.camera_K is None:
            return

        with self._depth_lock:
            depth_image = self._latest_depth
        if depth_image is None:
            depth_image = np.zeros((480, 640), dtype=np.float32)

        rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

        key_raw = cv2.waitKeyEx(1)
        key = key_raw & 0xFF if key_raw != -1 else -1

        # ---------- Calibration mode ----------
        if self._calib_active:
            self._calib_latest_frame = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)
            self._render_calib_ui(key)
            if key == 27:
                self._exit_calib_mode()
            return

        # r/g/b/y -> select target color
        color_key_map = {
            ord('r'): 'red',
            ord('g'): 'green',
            ord('b'): 'blue',
            ord('y'): 'yellow',
        }
        if key in color_key_map:
            self._target_color = color_key_map[key]
            self._tracked_blocks.clear()
            self.get_logger().info(
                f"Target color selected: {self._target_color.upper()}"
            )

        # 'c' -> enter LAB calibration (requires color selected first)
        if key == ord('c'):
            if self._target_color is not None:
                self._calib_latest_frame = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)
                self._enter_calib_mode(self._target_color)
            else:
                self.get_logger().warn(
                    "No target color selected. "
                    "Press r/g/b/y first, then 'c' to calibrate."
```

```
                )
            return

        # ESC -> exit
        if key == 27:
            self.get_logger().info("ESC pressed, shutting down...")
            raise KeyboardInterrupt

        # ---------- Shape detection by color mask ----------
        blocks, mask_vis = self._detect_shapes_by_color(rgb_image)

        # Temporal stabilization: remove per-frame shape flicker
        blocks = self._stabilize_blocks(blocks)

        # Calculate volume for each block (shape always kept for display)
        measured_blocks = []
        for block in blocks:
            if block['shape'] == 'None':
                continue
            out = {
                'shape': block['shape'],
                'center_px': block['center'],
                'contour': block['contour'],
                'rect': block['rect'],
                'volume_cm3': None,
                'height_mm': None,
            }
            vol = self._calculate_volume(block, depth_image)
            if vol is not None:
                out.update(vol)
            measured_blocks.append(out)

        self._latest_blocks = measured_blocks

        # ---------- Drawing ----------
        self._draw_results(rgb_image, measured_blocks, mask_vis)

    # ================================================================
    #  Drawing
    # ================================================================

    def _draw_results(self, rgb_image, blocks, mask_vis):
        """Draw shape contours, center points, and volume/height info at top-left"""
        result_image = rgb_image.copy()

        # Shape -> display color (RGB) mapping
        shape_color_map = {
            'Square': (255, 0, 0),      # blue
            'Rectangle': (0, 165, 255), # orange
            'Circle': (0, 255, 0),      # green
        }

        # ---- Top-left status line (y=25) ----
```

```
        if self._target_color is None:
            cv2.putText(result_image,
                        "Press r/g/b/y to select color | c=calib | ESC=exit",
                        (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.45,
                        (200, 200, 200), 1)
        else:
            cv2.putText(result_image,
                        f"Color: {self._target_color.upper()} | c=calib | ESC=exit",
                        (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.45,
                        (0, 255, 255), 1)

        # ---- Top-left: volume and center height for each block (y=55, 75, 95...) ----
        y_offset = 55
        LINE_H = 20

        if len(blocks) == 0:
            if self._target_color is not None:
                cv2.putText(result_image, "No block detected",
                            (10, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                            (0, 0, 255), 1)
        else:
            for block in blocks:
                shape_name = block['shape']
                color = shape_color_map.get(shape_name, (255, 255, 255))

                # Draw contour
                cnt = block['contour']
                cv2.drawContours(result_image, [cnt], -1, color, 2)

                # Draw min-area rectangle box
                box = cv2.boxPoints(block['rect'])
                box = np.int0(box)
                cv2.drawContours(result_image, [box], 0, color, 1)

                # Draw center point
                cpx, cpy = block['center_px']
                cv2.circle(result_image, (cpx, cpy), 4, color, -1)
                cv2.circle(result_image, (cpx, cpy), 6, (255, 255, 255), 1)

                # Shape label near center
                cv2.putText(result_image, shape_name,
                            (cpx + 10, cpy - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

                # Volume & height info at top-left
                if block['volume_cm3'] is not None:
                    info = (
                        f"{shape_name} | "
                        f"Vol:{block['volume_cm3']:.1f}cm3 | "
                        f"H:{block['height_mm']:.1f}mm"
                    )
                else:
                    info = f"{shape_name} | no depth/TF"
```

```
                cv2.putText(result_image, info, (10, y_offset),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.45, color, 1)
                y_offset += LINE_H

        # Show LAB mask side by side
        if mask_vis is not None and self._target_color is not None:
            cv2.putText(mask_vis, "LAB Mask", (10, 25),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)
            display = np.hstack((result_image, mask_vis))
        else:
            display = result_image

        display_bgr = cv2.cvtColor(display, cv2.COLOR_RGB2BGR)
        try:
            cv2.imshow("result_image", display_bgr)
        except Exception as e:
            if not self._display_error_logged:
                self.get_logger().warn(f"Display failed: {e}")
                self._display_error_logged = True
```

- LAB parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

- The LAB parameter is a 6-element array in the format `[L_min, a_min, b_min, L_max, a_max, b_max]`, defining the detection range of each color in the LAB color space
    - `L`: lightness component (0~255)
    - `a`: green-red component (0~255)
    - `b`: blue-yellow component (0~255)
- `blue_lab`: LAB detection range for the blue wooden block
- `green_lab`: LAB detection range for the green wooden block
- `red_lab`: LAB detection range for the red wooden block
- `yellow_lab`: LAB detection range for the yellow wooden block
- If the color recognition is inaccurate, you can press the `c` key in the window to enter the calibration interface to adjust the LAB range, and press `s` to save

```
/**:
  ros__parameters:
    .......
    blue_lab: [88,117,83,126,132,113]
    green_lab: [105,89,134,143,100,143]
    red_lab: [108,90,134,131,98,141]
    yellow_lab: [108,90,134,131,98,141]
 .....
```